HDR Bloom Rendering for a Real-Time Solar System Simulation

3D Graphics Project

Niccolò Benedetto Paolo Alberto Bordis

Polytech - Université Libre de Bruxelles



June 2026

Instructors: Prof. Daniele Bonatto, Eline Soetens

Introduction

- **Project Scope**: Development of a real 3D Solar System scene simulating a visually coherent representation of the Sun, the eight planets, Earth's moon, orbital trajectories, and an asteroid field.
- **Core Feature**: The main visual feature is the **Bloom rendering pipeline**.
- **Visual Concept**: The Sun acts as the primary emissive light source, producing an apparent glow effect in the final scene through High Dynamic Range (HDR) rendering.
- **Additional Features**: Textured Earth night lights, cubemap skybox, reflection mapping, object (.obj) rendering, free camera navigation, and game logic.

Requirements and Dependencies

The project relies entirely on modern Graphics APIs and C++ libraries:

- **Core Rendering**: OpenGL 3.3 Core Profile in C++.
- **Window & Input**: GLFW.
- **Function Loading**: GLAD.
- **Mathematics**: GLM.
- **User Interface**: Dear ImGui (runtime graphical controls).
- **Model Loading**: TinyObjLoader (for OBJ models).

Source Code: `https://github.com/NiccoBene00/OpenGL-SolarSystemSceneRendering`

Why Bloom?

Real light sources do not appear as perfectly sharp objects [1] - Chapter 12

- **Causes**
  - Bright regions bleed into adjacent pixels.
  - Camera sensors saturate near intense light sources.
  - Human vision perceives a glow around high-intensity emitters.

Objective: Reproduce the apparent glow of the Sun in real time.

Note: Bloom is not physically accurate but perceptually plausible [1].



Figure: Bloom effect - adapted from [1]

Bloom Pipeline

- Bloom is an image-space post-processing effect, applied after scene rendering.
- Its implementation is based on a multi-pass rendering architecture and HDR lighting information.
- It allows us to analyze the entire OpenGL pipeline.

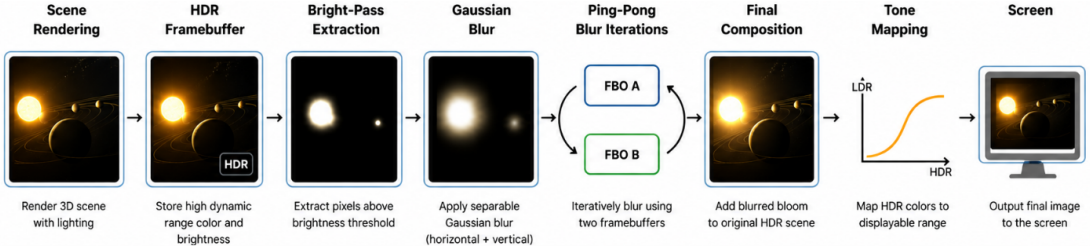


Figure: Entire Bloom Pipeline

HDR Framebuffer

Why not a standard framebuffer?

- **Normal Rendering**: A standard framebuffer stores color values within the range $[0, 1]$.
  - We cannot distinguish bright sources.
- **HDR Rendering**: Allows values > 1.0; this is essential for a realistic bloom effect.
- **Implementation**: Each color channel is stored using a 16-bit floating point format (`GL_RGBA16F` format) $\rightarrow$ a threshold can be applied to separate bright sources.

Bright-Pass Extraction

The fragments that will contribute to the bloom effect were isolated by considering a luminance threshold.

Luminance Calculation

$$\text{brightness} = 0.2126R + 0.7152G + 0.0722B$$

- Only fragments where the `brightness` exceeds the threshold were taken into account.

Implementation: The HDR framebuffer is split into two attachments:

- Color attachment 0 for the normal scene.
- Color attachment 1 for the bright fragments.

During rendering, the shader writes to the color attachments concurrently.

Gaussian Blur

The blur weights neighboring pixels using a Gaussian distribution.

Gaussian Function

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

Implementation: Instead of applying an $N \times N$ kernel in one single pass, two additional framebuffers are used (gaussian function is separable):

- Horizontal blur across x (Ping FBO framebuffer).
- Vertical blur across y (Pong FBO framebuffer).
- Complexity $O(N^2) \rightarrow O(2N)$.

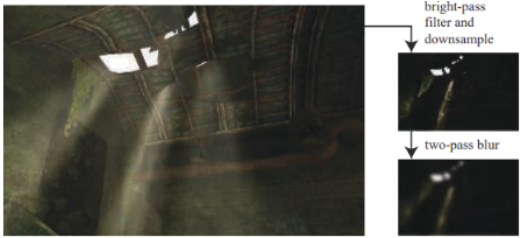


Figure: Example of Gaussian blur adapted from [1]

Final Bloom Composition and Tone Mapping

The final scene is rendered using the following fragment composition. By considering $C = (R, G, B)$:

$$C_{Final} = C_{Original} + C_{Bloom}$$

The last step involves converting HDR to displayable colors (tone mapping):

$$C_{LDR} = 1 - e^{-C_{HDR_{final}} \cdot exposure}$$

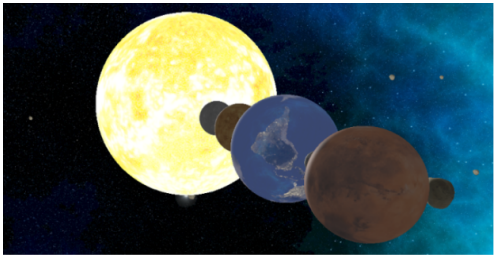


Figure: Without bloom

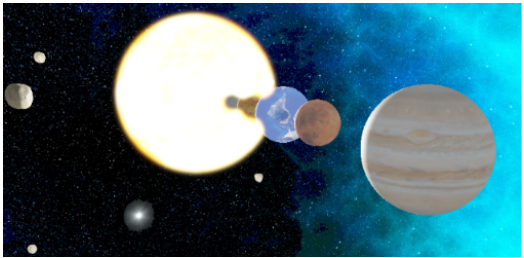


Figure: With bloom

Additional features

In order to make our scene more realistic, additional features have been implemented...

Phong Lighting Model

All non-emissive objects (planets, asteroids) in the scene are illuminated using the Phong lighting model [2] (a simplification respect to Blinn–Phong model presented in [1]).

Total Illumination

$$I = I_{ambient} + I_{diffuse} + I_{specular}$$

Diffuse Component (Lambert Cosine Law)

$$I_{diffuse} = \max(\mathbf{N} \cdot \mathbf{L}, 0)$$

Specular Reflection

$$I_{specular} = \max(\mathbf{R} \cdot \mathbf{V}, 0)^{\text{shininess}}$$

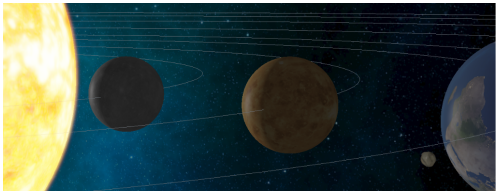


Figure: Phong illumination to discriminate daylight from dark side

Skybox and Cubemap Environment

Goal: Create the perception of an infinite universe space environment.

Implementation

- Cubemap surrounding the observer composed of six textures.
- Camera translation removed in order to create an infinite distance illusion. Only rotation is evaluated for the View Matrix (3×3 matrix):

$$View_{Skybox} = glm :: mat4(glm :: mat3(view))$$

Visual Result: The observer can rotate freely while the background remains infinitely distant.

Environment Mapping Reflections

Goal: The skybox is used to simulate a reflective Earth surface.

Implementation

- Compute the incident view direction (I) from the camera to the reflective surface (Earth).
- Compute the reflection vector (R) with respect to the surface normal (N) of the reflective surface.
- Sample the cubemap along that direction to obtain the color to reflect.

Reflection Equation

$$R = I - 2(N \cdot I)N$$

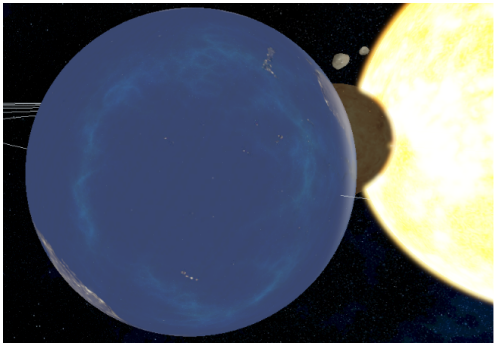


Figure: Reflection of the skybox to Earth's surface

External Models and Camera System

Goal: Render more complex objects (asteroids) instead of only simple sphere meshes (planets).

- Tiny Object Loader was used to extract vertices, normals, and texture coordinates.
- Data was uploaded into GPU through a VBO (Vertex Buffer Object) which is described by a VAO (Vertex Array Object).
- The model was rendered via the normal OpenGL pipeline.

Goal: Make the scene interactive with the observer.

- FPS camera and WASD system.
- A view matrix that simulates the desired position of the camera is computed.
- Keyboard input updates camera position and orientation, from which the View matrix is recomputed each frame

Interactive Controls

Goal: Make the scene interactive by using a GUI.

- Bloom toggle
- Exposure control
- Time scale
- Orbit visibility

Implementation: C++ variables that dynamically manipulate the scene are connected to the graphical widgets implemented by the `ImGui` library.

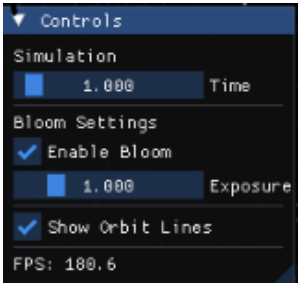


Figure: ImGui controls

References I

Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michał Iwanicki, and Sébastien Hillaire.
Real-Time Rendering, Fourth Edition.
CRC Press, 2021.

Michael Ashikhmin and Peter Shirley.
An anisotropic phong light reflection model.
Technical Report UUCS-00-014, Department of Computer Science, University of Utah, June 2000.